

Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)

Terminology

- A **tree** is a collection of nodes.
- A tree consists of a distinguished node r , called the **root**, and zero or more (sub)trees $T_1, T_2, \dots, T_k$, each of whose roots are connected by a directed edge to r .
- The root of each subtree is said to be a *child* of r , and r is the *parent* of each subtree root

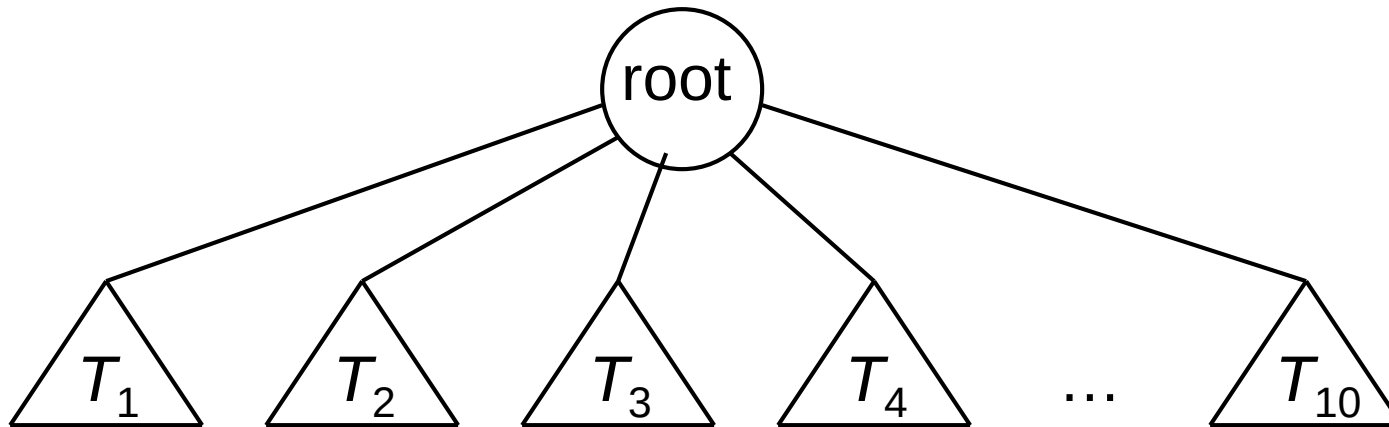


Figure 4.1 Generic tree

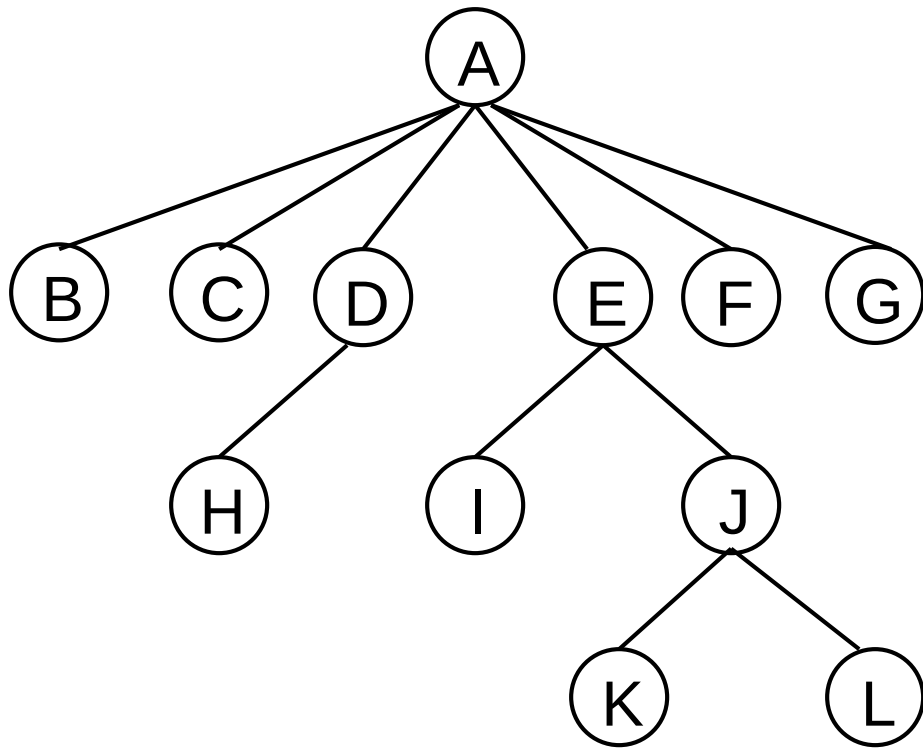
Terminology

- The root of each subtree is said to be a **child** of r and r is said to be the **parent** of each subtree root.
- **Leaves**: nodes with no children (also known as external nodes)
- **Internal Nodes**: nodes with children
- **Siblings**: nodes with the same parent
- A **path** from node n_1 to n_k is defined as a sequence of nodes $n_1, n_2, \dots, n_k$ such that n_i is the parent of n_{i+1} for $1 \leq i \leq k$.
- The length of this path is the number of edges on the path namely $k-1$.
- The length of the path from a node to itself is 0.
- There is exactly one path from the root to each node.

Terminology

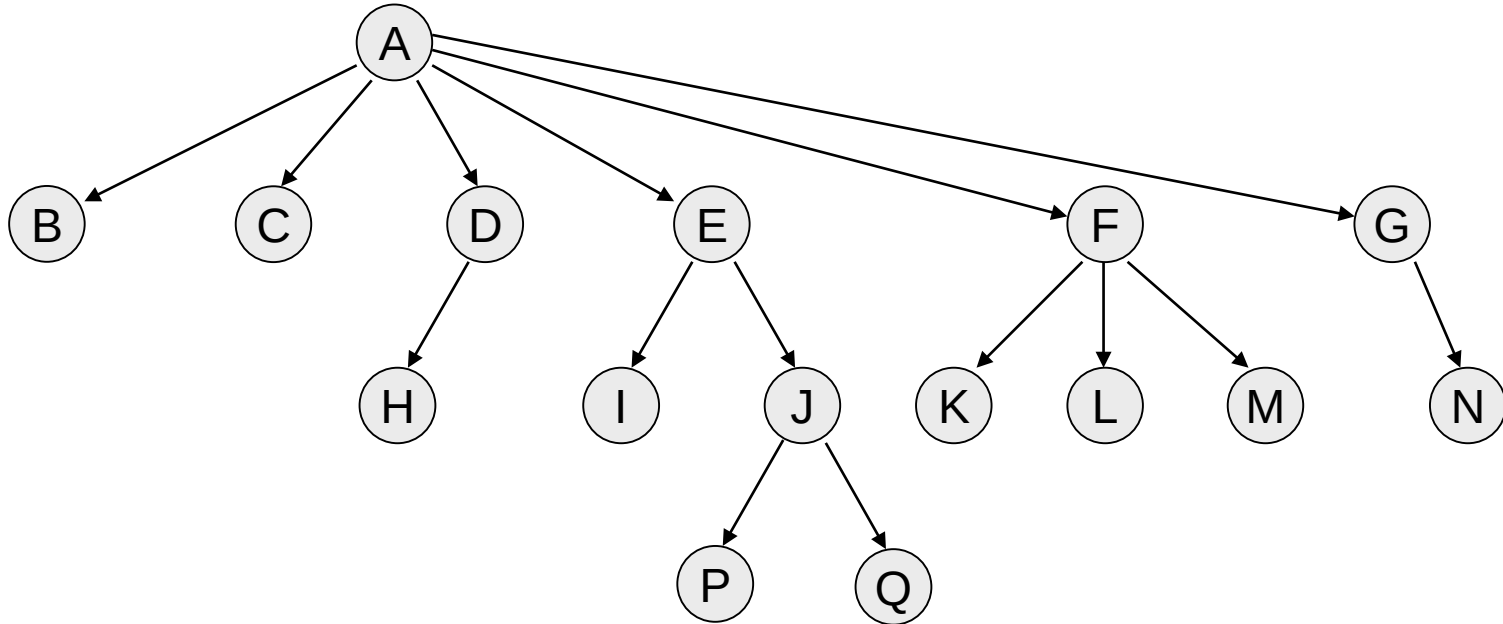
- The *depth* of node n_i is the length of the path from *root* to node n_i
- The *height* of node n_i is the length of longest path from node n_i to a *leaf*.
- If there is a path from n_1 to n_2 , then n_1 is *ancestor* of n_2 , and n_2 is *descendent* of n_1 .
 - If $n_1 \neq n_2$ then n_1 is *proper ancestor* of n_2 , and n_2 is *proper descendent* of n_1 .

Example



- The root is *A*.
- Node *E* has *A* as a **parent** and *I*, *J* as **children**.
- Each node may have an arbitrary number of children, possibly zero.
- Nodes with no children are known as **leaves**.
- Nodes with the same parent are **siblings**.
- **Grandparent** and **grandchild** relations can be defined in a similar manner.

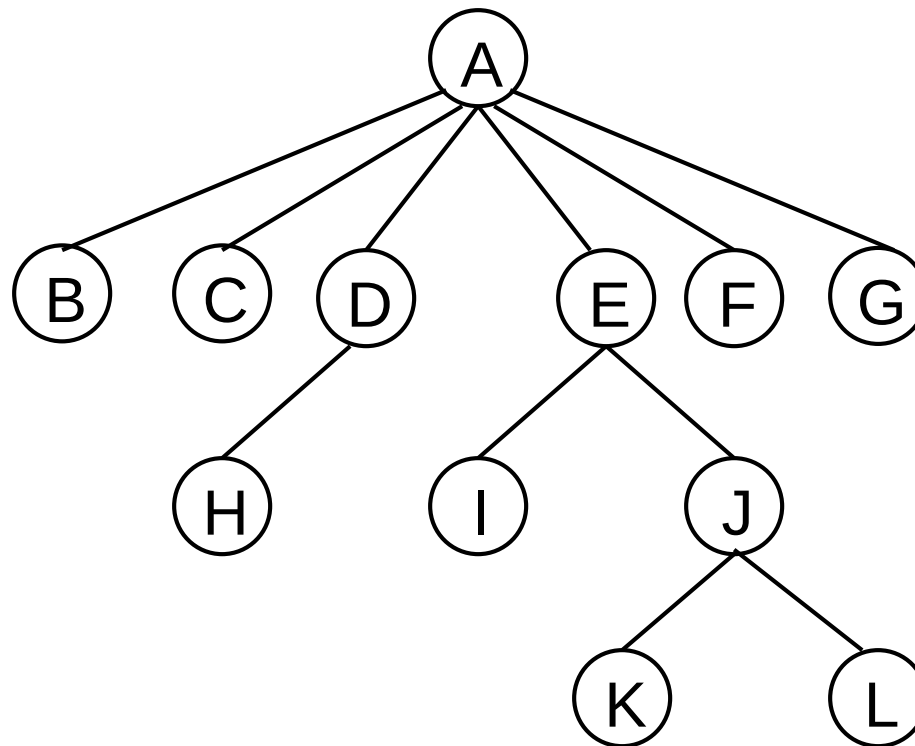
Example



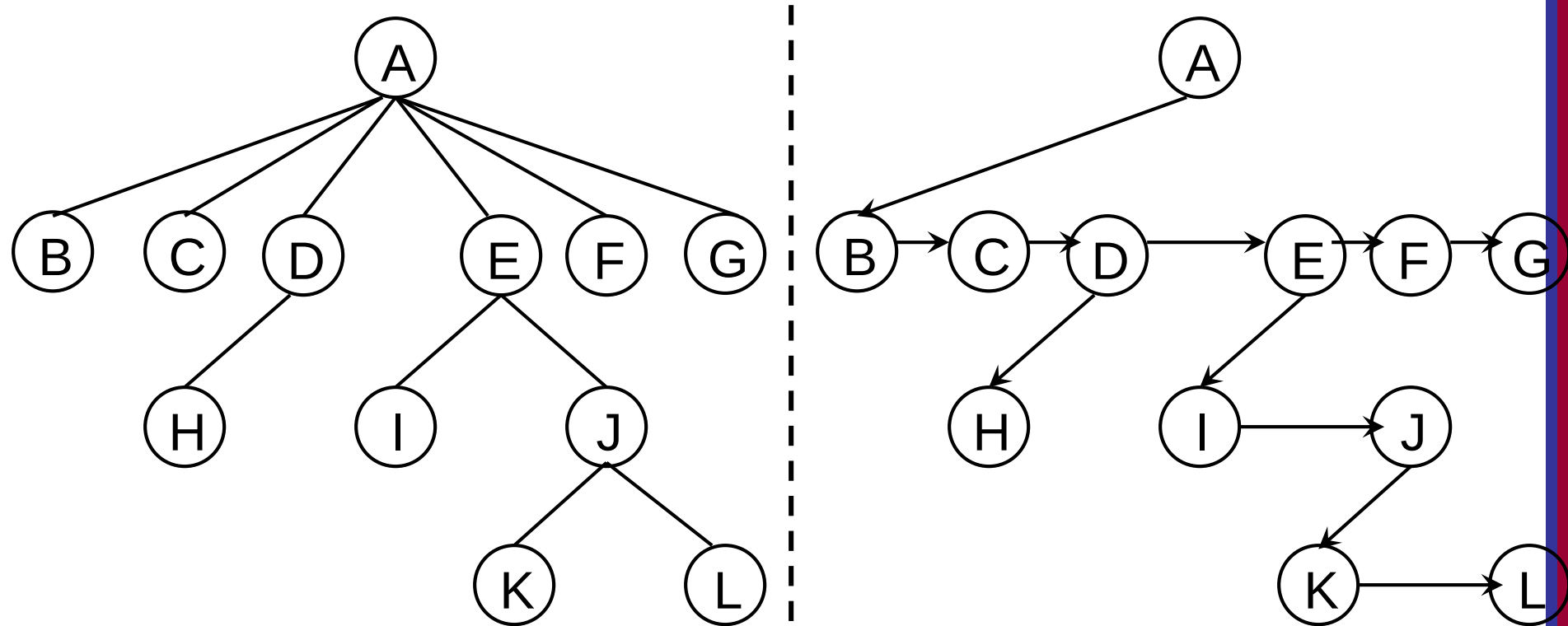
- Node *A* has 6 children: *B, C, D, E, F, G*.
- *B, C, H, I, P, Q, K, L, M, N* are leaves in the tree above.
- *K, L, M* are siblings since *F* is parent of all of them.

Example

- *E* is at depth 1 and height 2;
- *D* is at depth 1 and height 1;
- The height of the tree is 3.



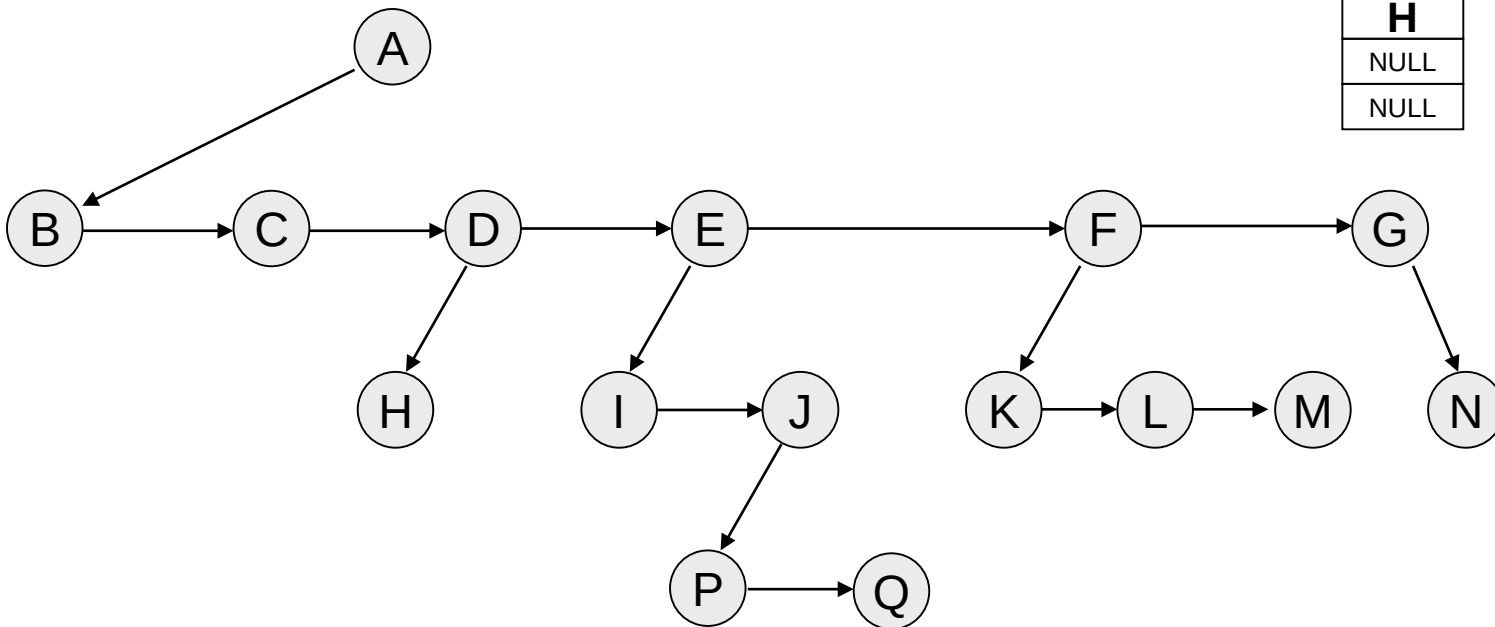
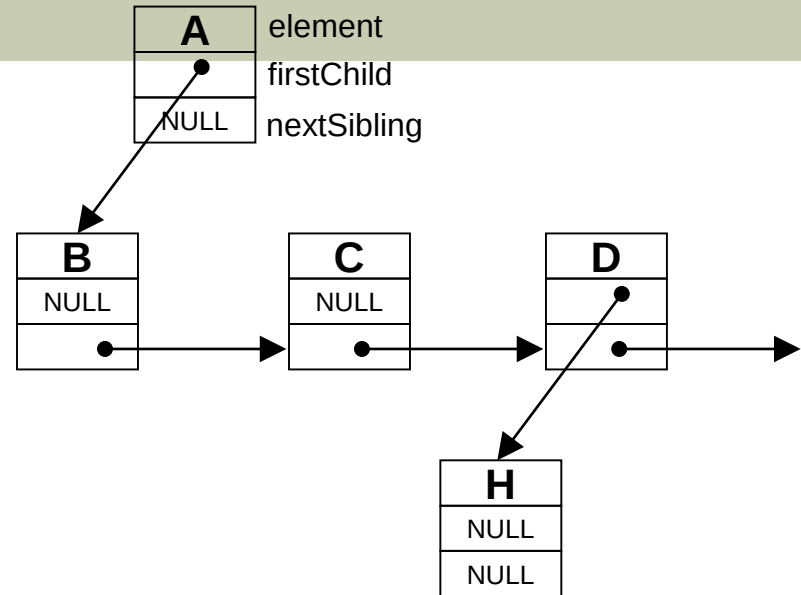
Implementation of Trees



- Arrows that point downward are *FirstChild* pointers.
- Arrows that go left to right are *NextSibling* pointers.
- Node *E* has both a pointer to a sibling (*F*) and a pointer to a child (*I*), while some nodes have neither.

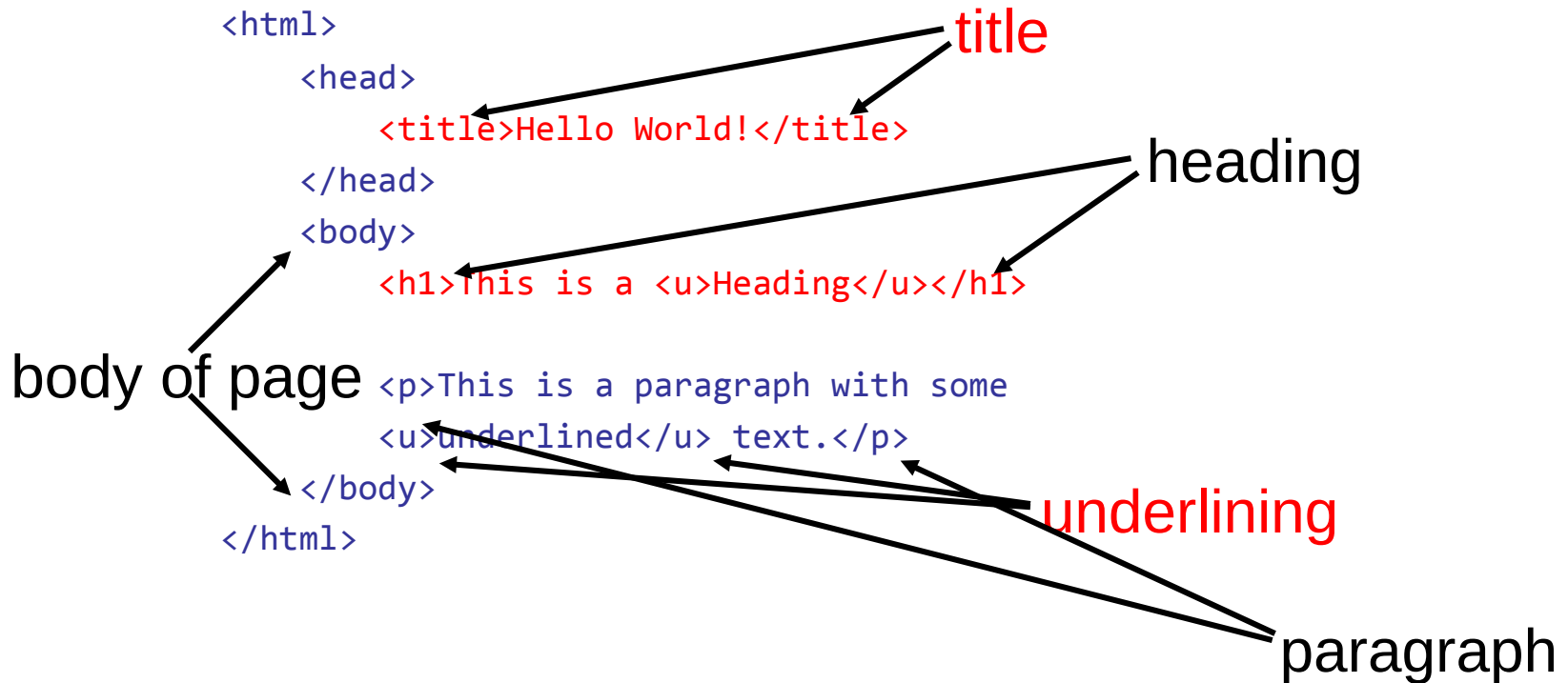
Implementation of Trees

```
struct TreeNode
{
    char Element;
    struct TreeNode *FirstChild;
    struct TreeNode *NextSibling;
};
```



Example: XHTML and CSS

Consider the following XHTML document

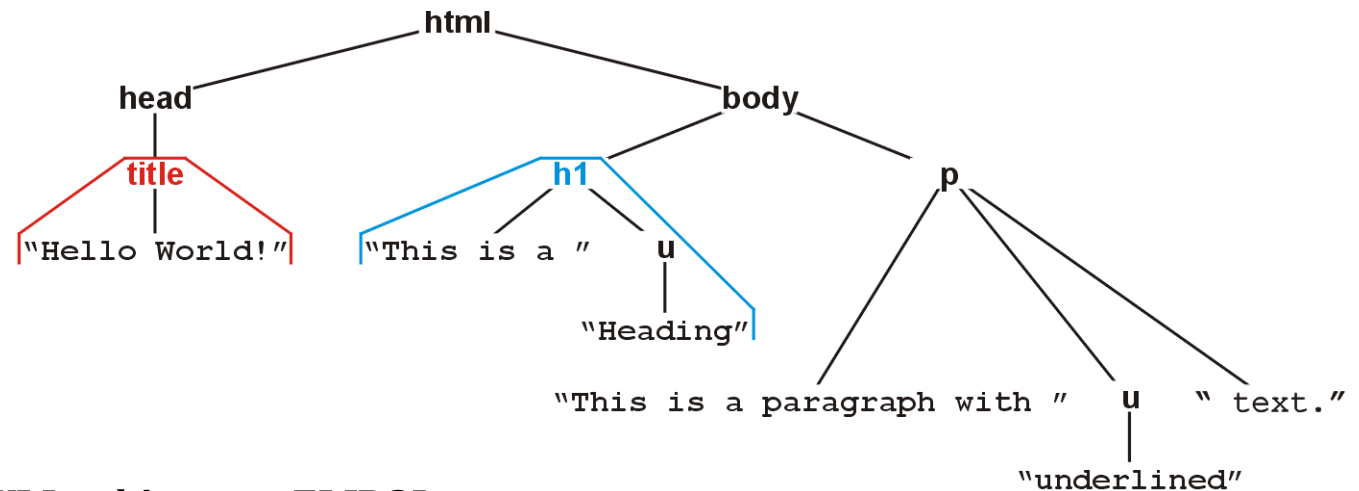


Example 1: XHTML and CSS

The nested tags define a tree rooted at the HTML tag

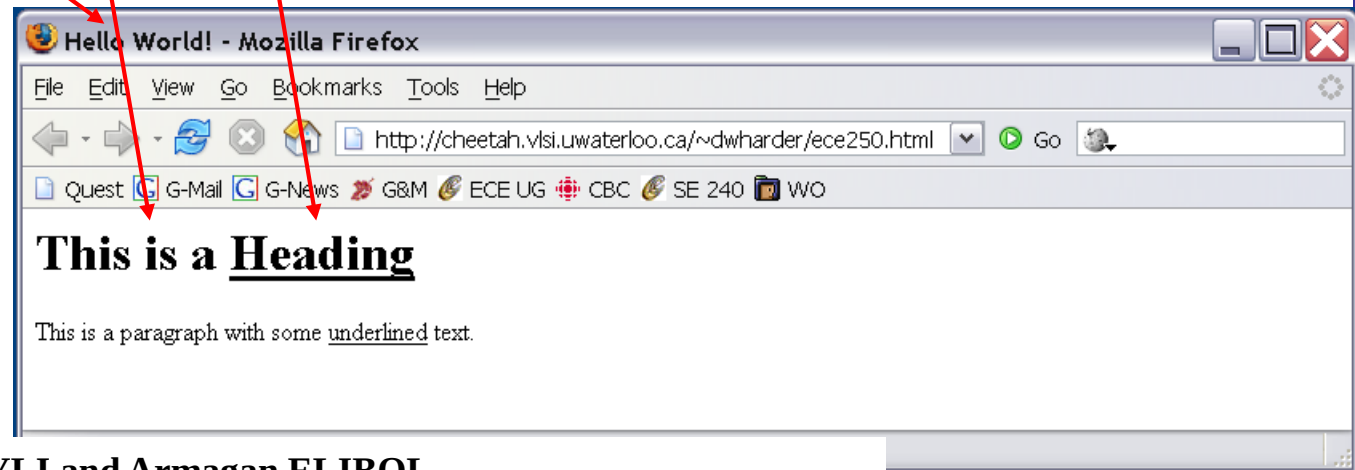
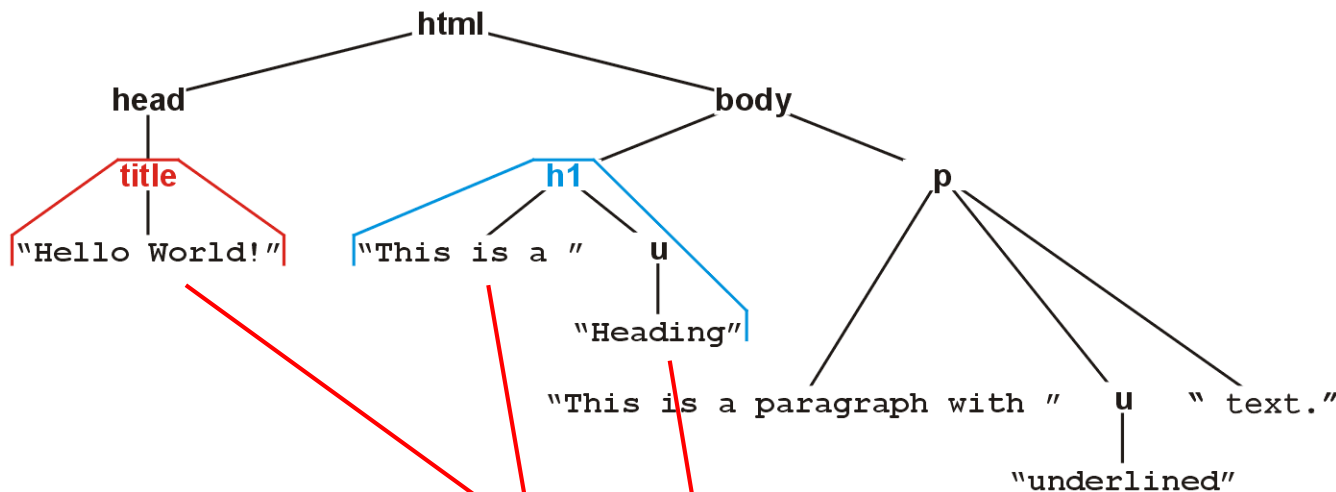
```
<html>
  <head>
    <title>Hello World!</title>
  </head>
  <body>
    <h1>This is a <u>Heading</u></h1>

    <p>This is a paragraph with some
    <u>underlined</u> text.</p>
  </body>
</html>
```



Example 1: XHTML and CSS

Web browsers render this tree as a web page



Example 2: UNIX OS

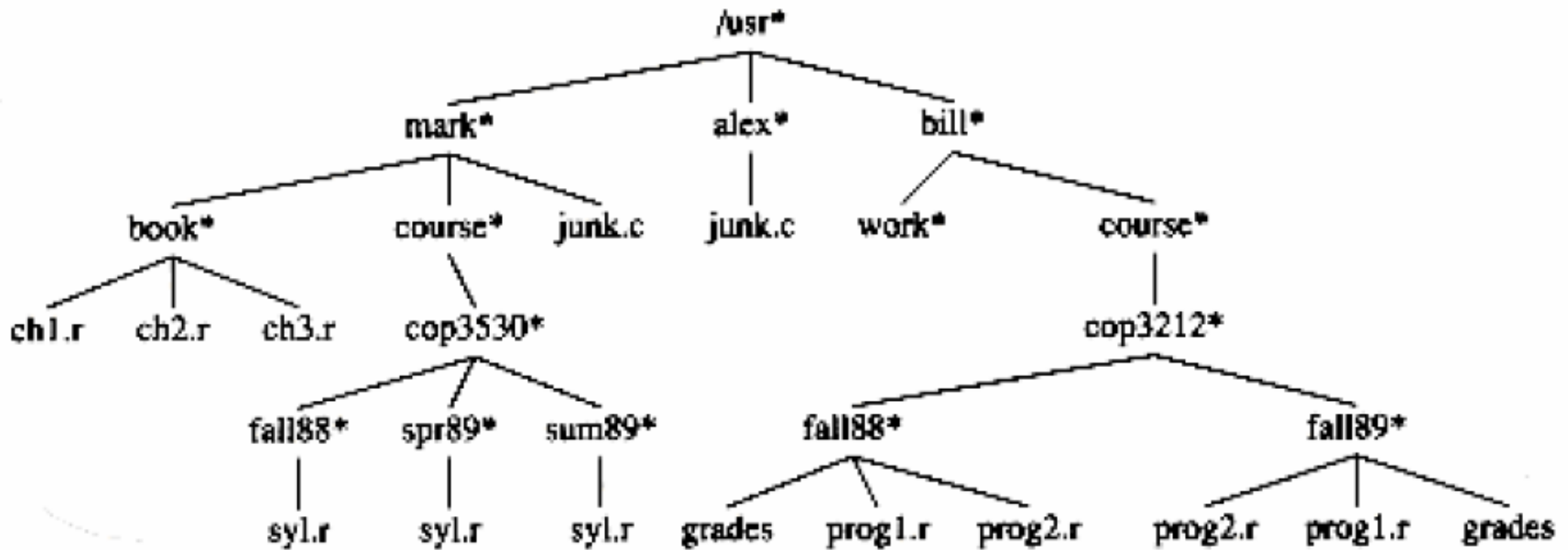


Figure 4.5 Unix Directory

Example 2: UNIX OS- List Directory - Preorder

```
void list_directory ( Directory_or_file D )
{
    list_dir ( D, 0 );
}

void list_dir ( Directory_or_file D, unsigned int depth )
{
    if ( D is a legitimate entry )
    {
        print_name ( depth, D );
        if( D is a directory )
            for each child, c, of D
                list_dir( c, depth+1 );
    }
}
```

Figure 4.6 Routine to list a directory in a hierarchical file system

Example 2: UNIX OS- List Directory-Result

```
/usr
  mark
    book
      ch1.r
      ch2.r
      ch3.r
    course
      cop3530
        fall05
          syl.r
        spr06
          syl.r
        sum06
          syl.r
      junk
    alex
      junk
    bill
      work
      course
        cop3212
          fall05
            grades
            prog1.r
            prog2.r
          fall06
            prog2.r
            prog1.r
            grades
```

Figure 4.7
The (preorder) directory listing

Example 2: UNIX OS- List Directory - Postorder

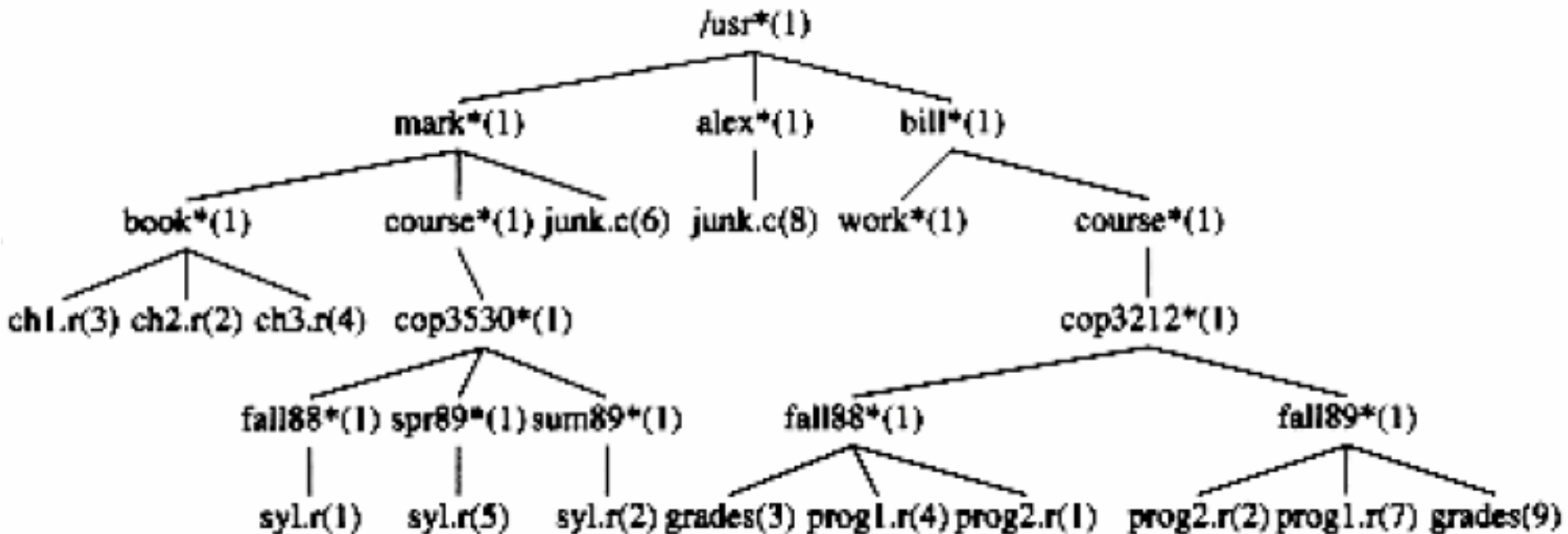


Figure 4.8

Unix directory with file sizes obtained via postorder traversal

Example 2: UNIX OS- Size of A Directory (Postorder)

```
unsigned int  size_directory( Directory_or_file D )
{
    unsigned int total_size;
    total_size = 0;
    if( D is a legitimate entry)
    {
        total_size = file_size( D );
        if( D is a directory )
            for each child, c, of D
                total_size += size_directory( c );
    }
    return( total_size );
}
```

Figure 4.9 Routine to calculate the size of a directory

Example 2: UNIX OS- Size of A Directory (Postorder)

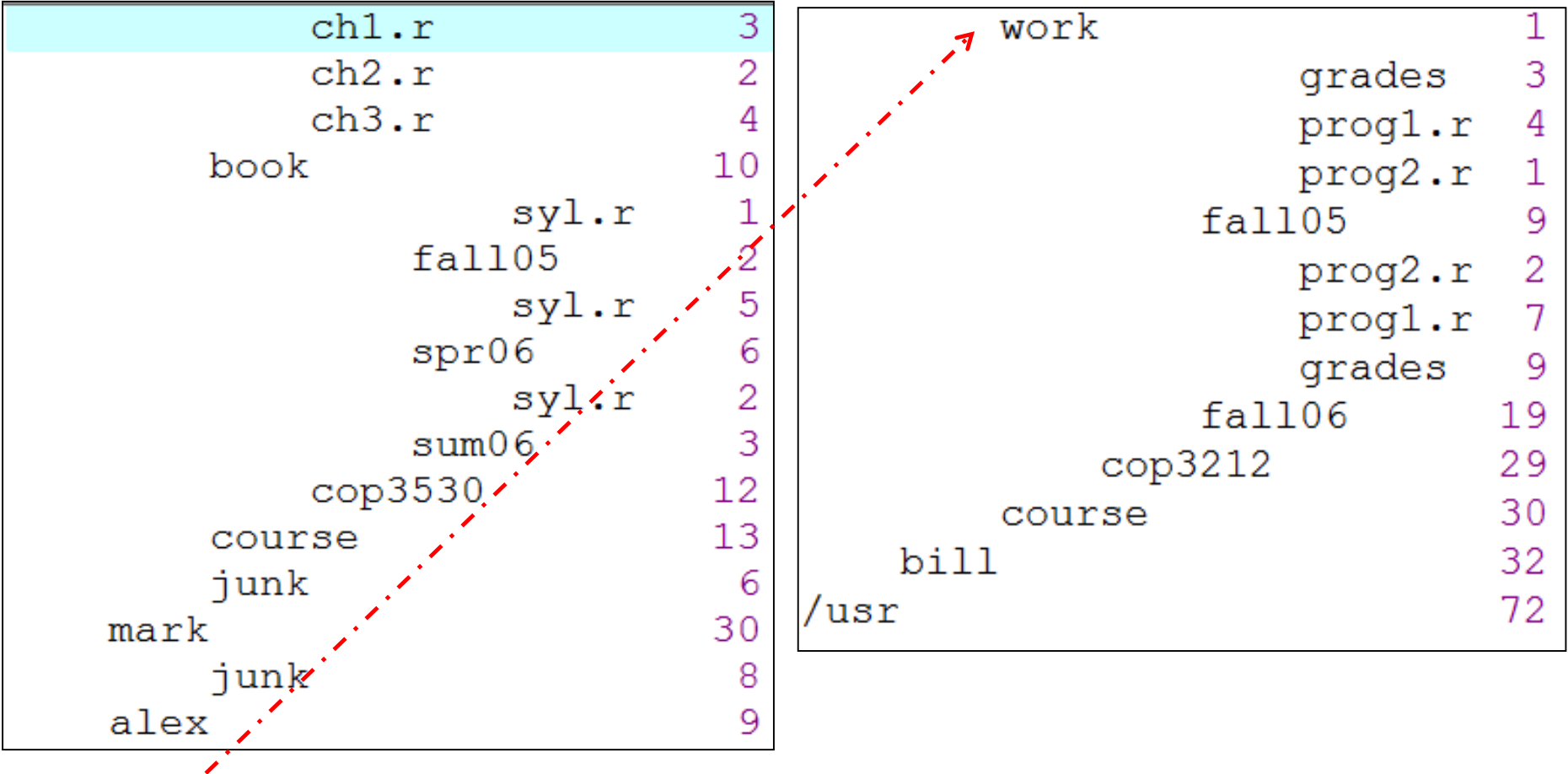
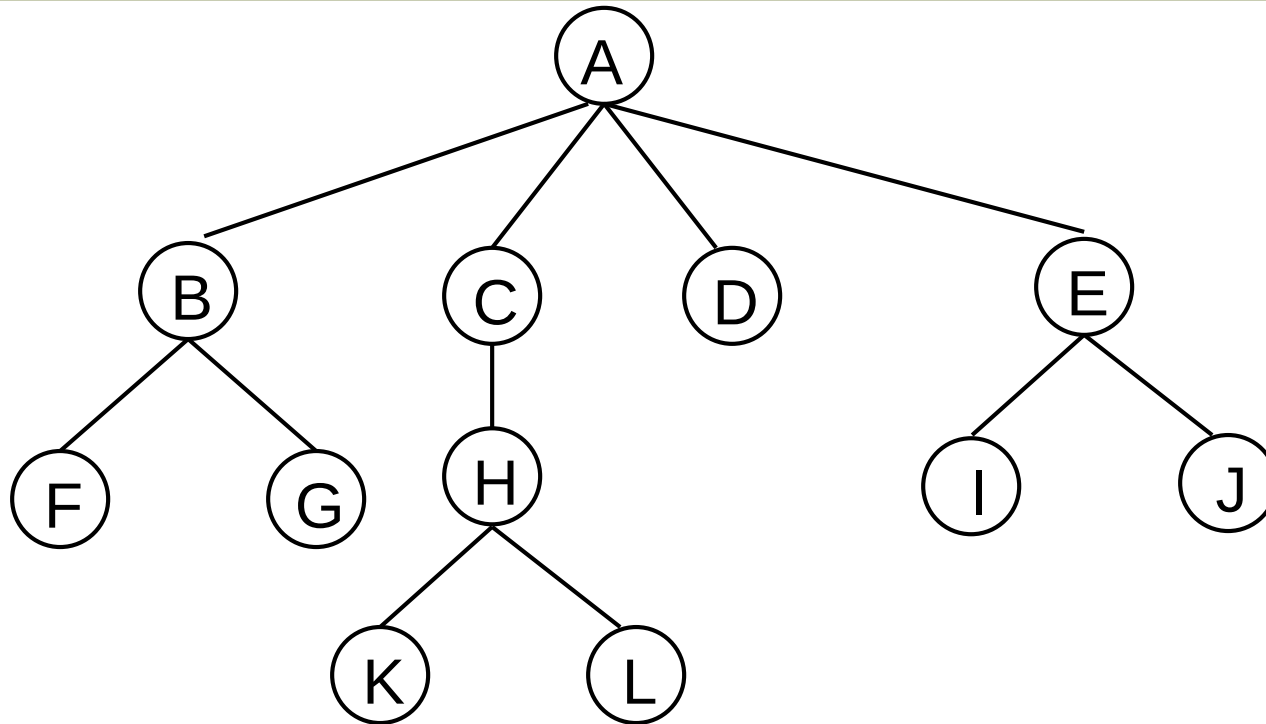


Figure 4.10 Trace of the size function

Expression Trees (Tree Traversal)

- The purpose of tree traversal: visit (perform some operations on) each node in a tree systematically:
 - ✓ **Preorder traversal:** The operations at a node are performed before its children from left to right are processed.
 - ✓ **Postorder traversal:** The operations at a node are performed after its children from left to right are processed.
- The operations on each node are performed recursively.

Example: Give the outputs of the preorder and postorder traversals of the following tree.



✓ **Preorder traversal** : A B F G C H K L D E I J

✓ **Postorder traversal** : F G B K L H C D I J E A

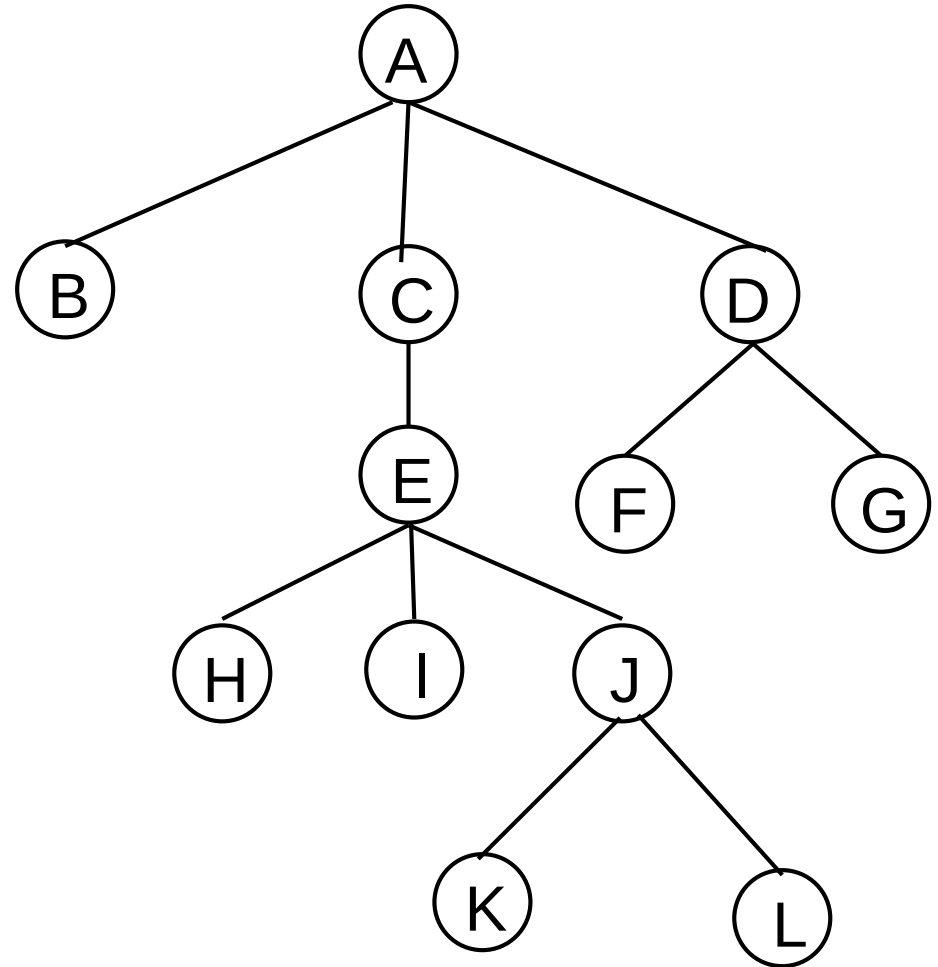
Example: Give the outputs of the preorder and postorder traversals of the given tree.

✓ **Preorder traversal:**

A B C E H I J K L D F G

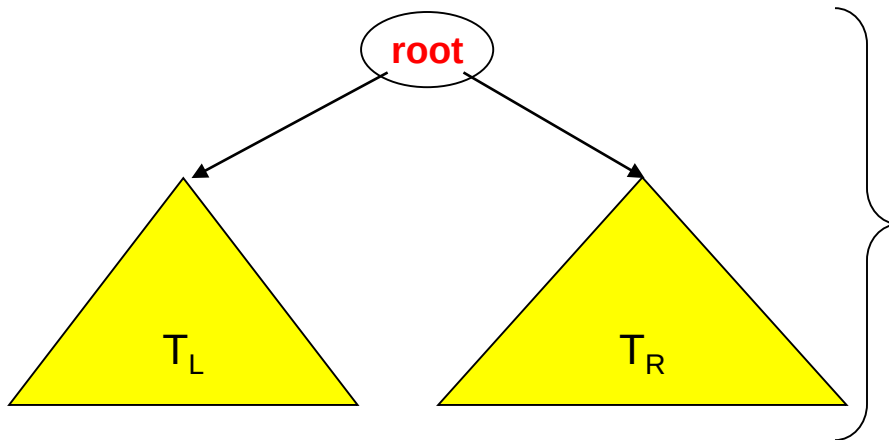
✓ **Postorder traversal:**

B H I K L J E C F G D A



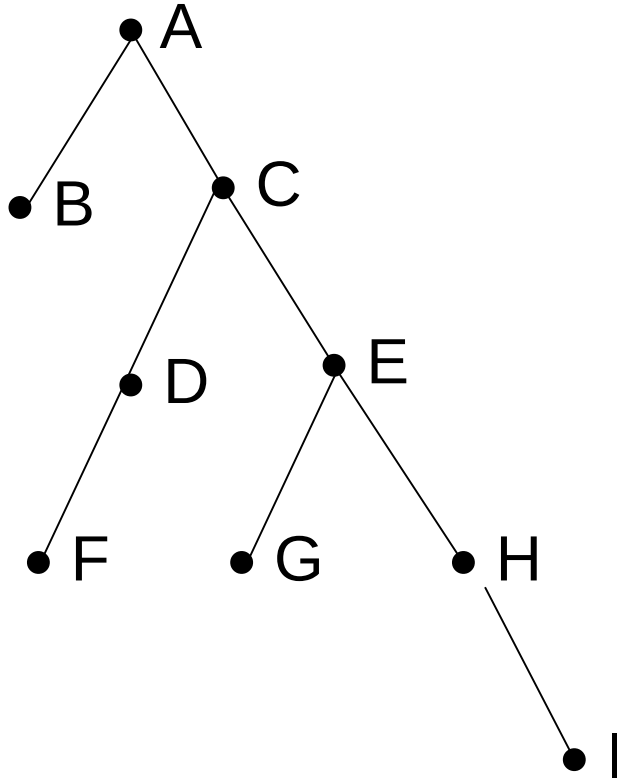
4.2. Binary Trees

- A *binary tree* is a tree in which no node can have more than two children
- The depth can be as large as $N-1$ in the worst case.



A binary tree consisting of a root and two subtrees T_L and T_R , both of which could possibly be empty.

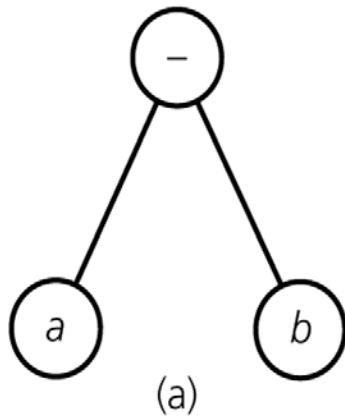
Example



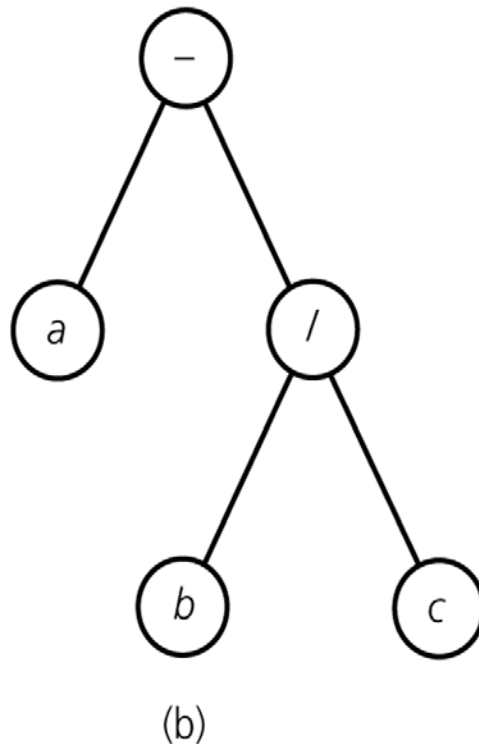
- A is the root
- B is the left child of A
- C is the right child of A
- D doesn't have a right child
- H doesn't have a left child
- B, F, G and I are leaves

Binary Tree - Representing Algebraic Expressions

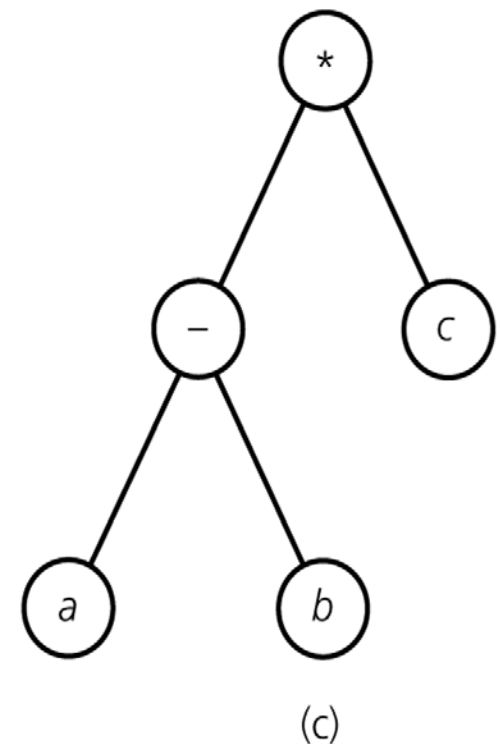
$a - b$



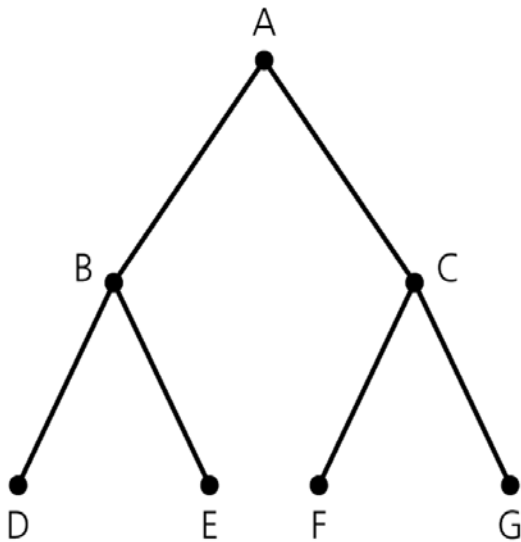
$a - b / c$



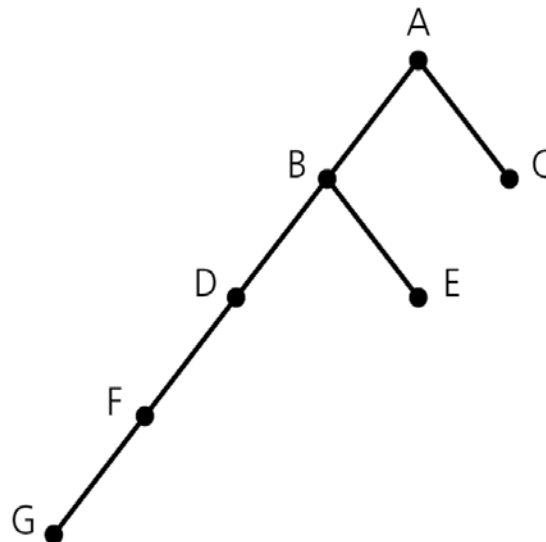
$(a - b) * c$



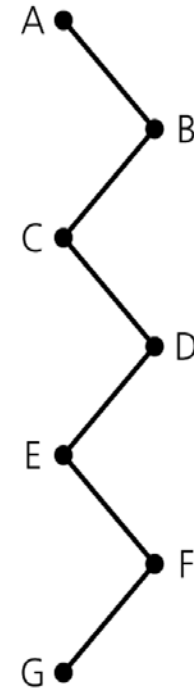
Height of Binary Tree (cont.)



(a)



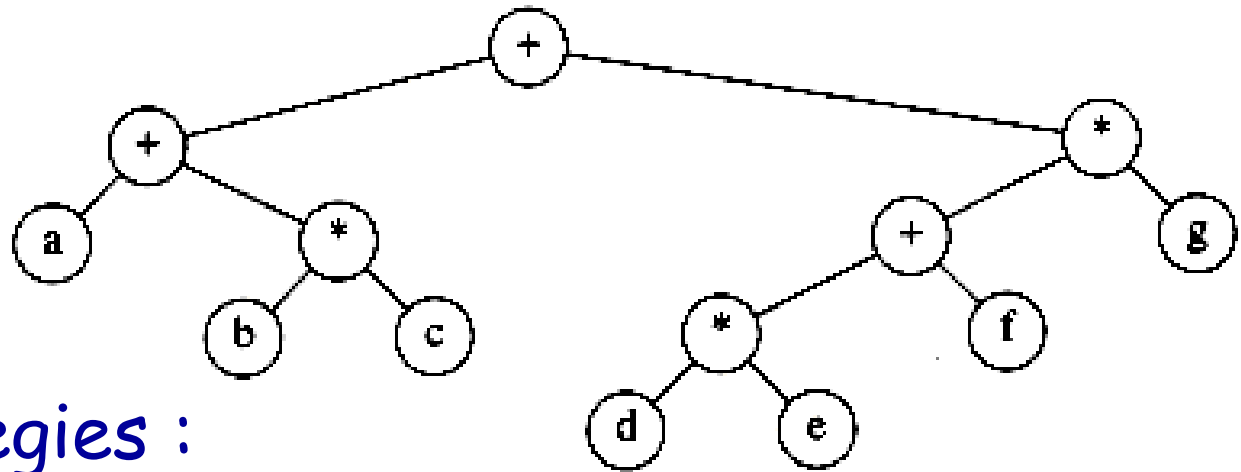
(b)



(c)

Binary trees with the same nodes but different heights

Expression Trees (Tree Traversal)



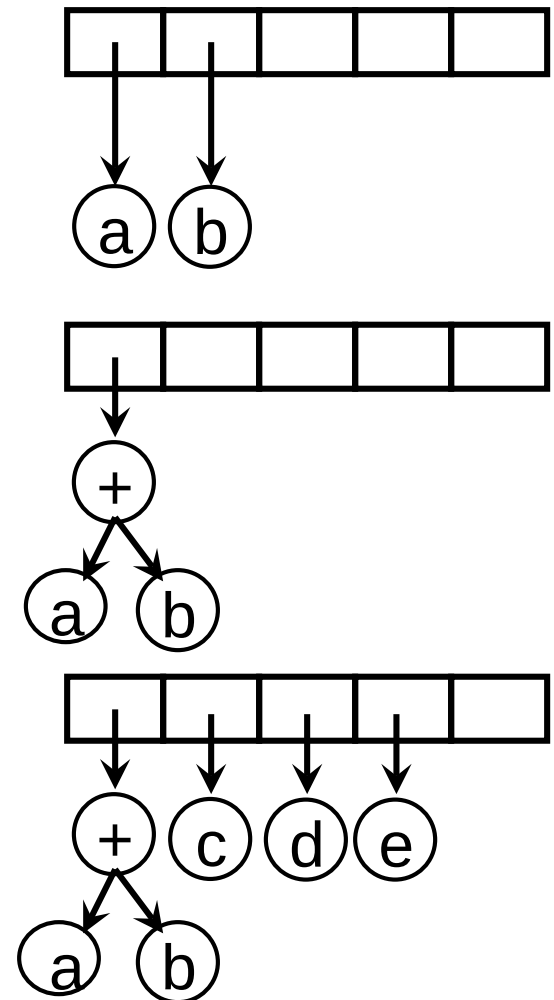
General strategies :

- *Inorder/infix* is (left, node, right):
 $(a + (b * c)) + (((d * e) + f) * g)$
- *Postorder/postfix* is (left, right, node):
 $a b c * + d e * f + g * +$
- *Preorder/prefix* is (node, left, right):
 $+ + a * b c * + * d e f g$

Constructing a Tree From Postfix Expression Using a Stack Structure

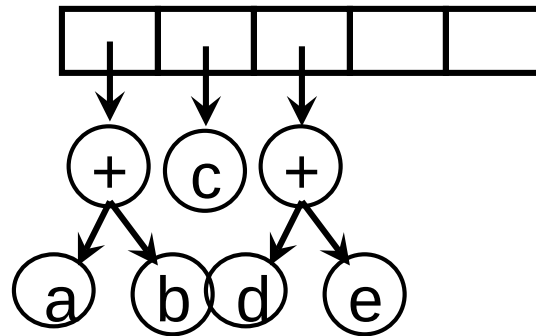
Input: a b + c d e + * *

- The first two symbols are operands, so we create one-node trees and push pointers to them onto a stack. For convenience, we will have the stack grow from left to right in the diagrams.
- Next, a '+' is read, so two pointers to trees are popped, a new tree is formed, and a pointer to it is pushed onto the stack.
- Next, c, d, and e are read, and for each a one-node tree is created and a pointer to the corresponding tree is pushed onto the stack.

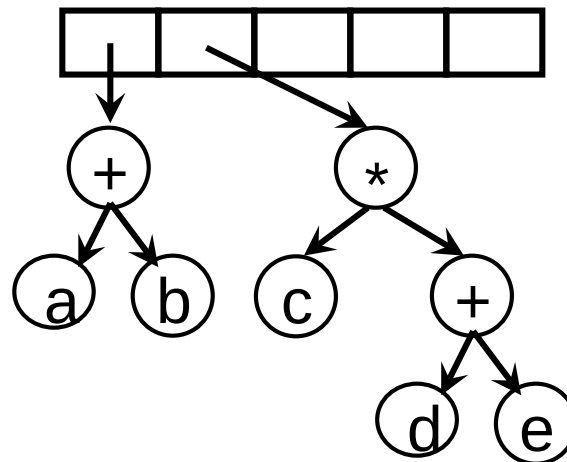


Constructing a Tree From Postfix Expression Using a Stack Structure

- Now a '+' is read, so two trees are merged.

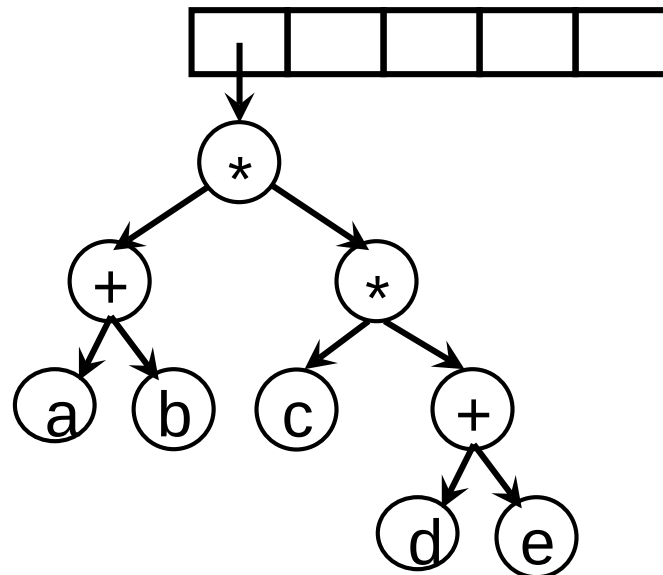


- Continuing, a '*' is read, so we pop two tree pointers and form a new tree with a '*' as root.



Constructing an Expression Tree

- Finally, the last symbol is read, two trees are merged, and a pointer to the final tree is left on the stack.



Algorithm Analysis And Data Structures

Questions and Answers

Thank you ...

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)